

Lecture 2 – Data Engineering for AI Systems

Building the Backbone of Reliable Machine Learning

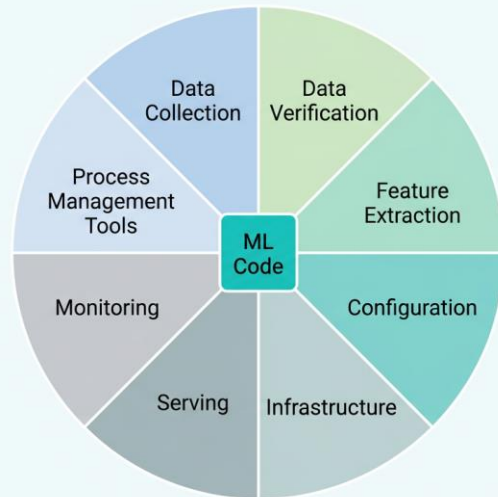
The "Hidden Technical Debt" of ML

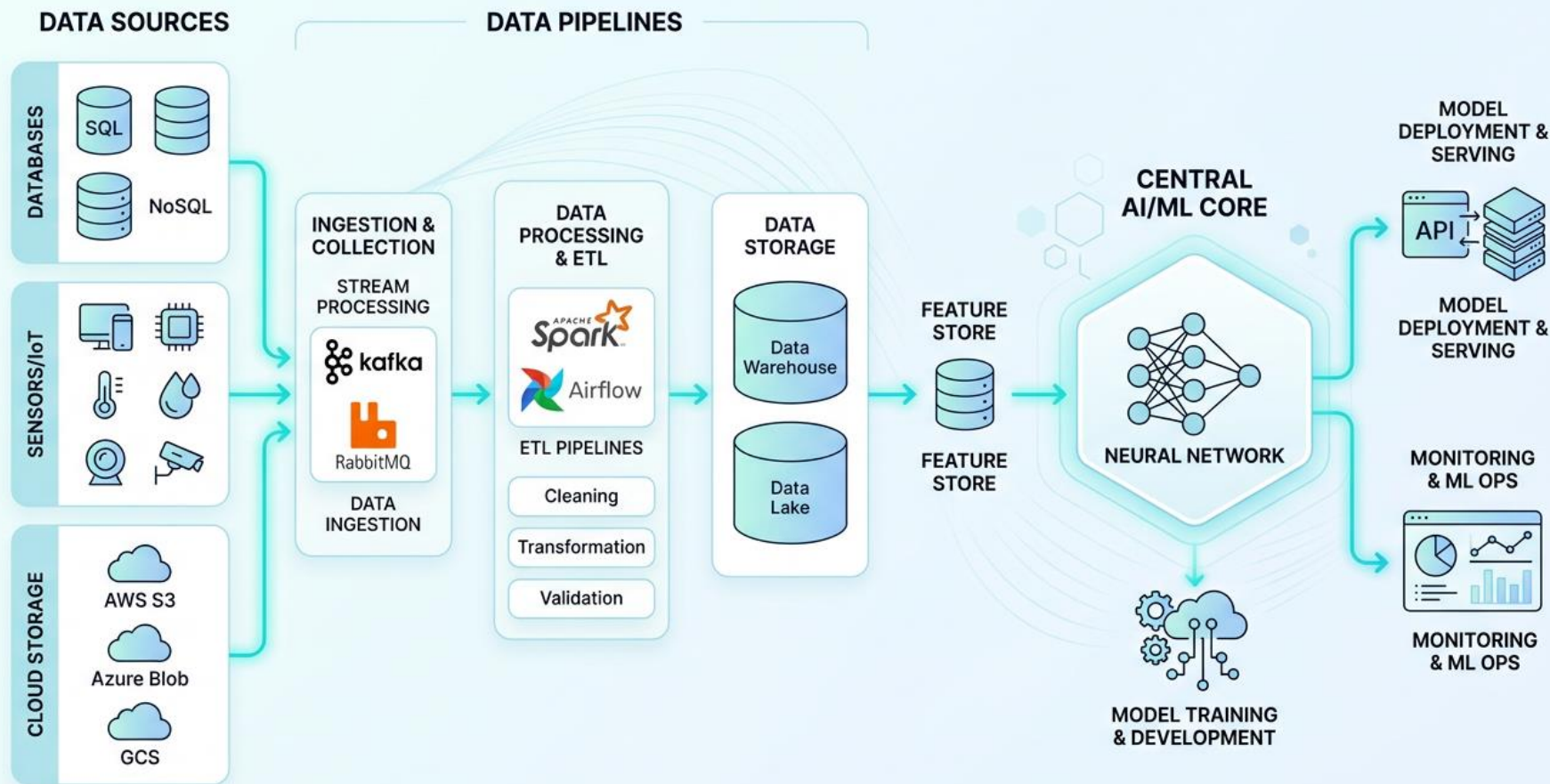
The Myth vs Reality

- **Myth:** AI systems are mostly ML code
- **Reality:** ML code is only ~5% of the system
- The rest: infrastructure, serving, data pipeline

Why Data Engineering Matters:

- Prevents "Garbage In, Garbage Out" (GIGO).
- Ensures models perform reliably over time.
- Enables scalability from megabytes on a laptop to terabytes in the cloud.





Data Ingestion – Batch vs Stream

Batch Processing

Processes large volumes at scheduled intervals

Tools: Apache Airflow, AWS Glue

Use Case: Training models

Stream Processing

Processes data continuously in real-time

Tools: Apache Kafka, AWS Kinesis

Use Case: Fraud detection, autonomous vehicles

Modern Data Pipelines – ETL vs ELT

ETL (Extract, Transform, Load)

Transform data before warehouse. Good for privacy rules.

ELT (Extract, Load, Transform)

Load raw data first, transform using cloud compute.

AI Advantage of ELT: Keeps raw data intact for future feature engineering

Data Preprocessing – Feature Scaling

Min-Max Scaling

$$X_{\text{norm}} = (X - X_{\text{min}}) / (X_{\text{max}} - X_{\text{min}})$$

Z-Score Standardization

$$Z = (X - \mu) / \sigma$$

Example: House price 500k with min=200k, max=1000k

$$X = (500000 - 200000) / (1000000 - 200000) = 0.375$$

The Danger of Outliers in Imputation

Dataset: 40k, 45k, 50k, [MISSING], 1000k (CEO)

✗ Mean Imputation

Sum: 1135k

Mean: 283.75k

Heavily skewed by outlier!

✓ Median Imputation

Sorted: 40, 45, 50, 1000

Median: 47.5k

Robust to outliers!

Silent Failures & Automated Validation

Bad inputs in AI cause silent failures, not crashes

```
from pydantic import BaseModel, Field, ValidationError

class UserDataInput(BaseModel):
    age: int = Field(..., ge=0, le=120) # Age must be 0-120
    income: float = Field(..., ge=0.0) # Income cannot be negative
    user_segment: str

# If bad data arrives in the pipeline, it raises an error BEFORE hitting the AI:
try:
    bad_data = UserDataInput(age=150, income=-500, user_segment="A")
except ValidationError as e:
    print("Pipeline halted: Data contract violated!")
```


Storage Formats – CSV vs Parquet

CSV (Row-Based)

- Human-readable
- Inefficient queries
- Must read entire file

Parquet (Columnar) ✓

- Machine-optimized
- Highly compressed
- Load only needed columns

Parquet is the industry standard for ML data lakes

Handling Sequential & Time-Series Data

Danger: Random Splits Cause Look-ahead Bias

Cannot train AI to predict the past using future data

Solution: Temporal Splitting

- Fold 1: Train Jan-Mar → Predict Apr
- Fold 2: Train Jan-Apr → Predict May
- Fold 3: Train Jan-May → Predict Jun

The AI Reproducibility Crisis & DVC

Model State = Code + Data + Environment

Git tracks code perfectly, but fails on 50GB datasets

```
# 1. Track massive dataset with DVC (hashes the data)
```

```
dvc add data/training_data.parquet
```

```
# 2. Track the lightweight DVC pointer with Git
```

```
git add data/training_data.parquet.dvc
```

```
git commit -m "Added Week 2 training dataset"
```

```
# 3. Push heavy data to cloud storage (S3/GCS)
```

```
dvc push
```

Summary & Next Steps

Key Takeaways

- **Architecture:** ELT is modern standard
- **Math:** Scale data, beware outliers
- **Storage:** Parquet > CSV for ML
- **Validation:** Use data contracts (Pydantic)

Week 3: Feature Engineering